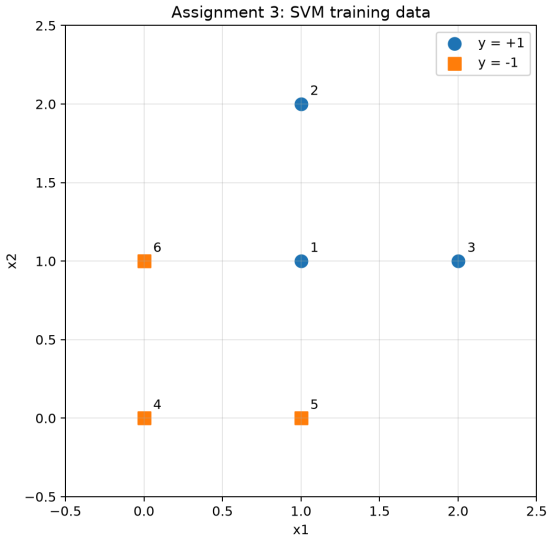
CSE 242 Assignment 3: SVMs, K-means, and K-NN

Name: Qijun Han

Question 1: Hard-margin linear SVM



(a) Support vectors: points 1 $(1,1)$, 5 $(1,0)$ and 6 $(0,1)$.

Every positive point has $x_1 + x_2 \geq 2$ and every negative point has $x_1 + x_2 \leq 1$, so the classes are separated along the $(1,1)$ direction. The positive point closest to the negative class is point 1. It is equally close to points 5 and 6, and its distance to the segment between them is $1/\sqrt{2}$, reached at $(0.5, 0.5)$. The widest gap is therefore between the line $x_1 + x_2 = 1$ through points 5 and 6 and the parallel line $x_1 + x_2 = 2$ through point 1, so these three points lie on the margin. Points 2 and 3 ($x_1 + x_2 = 3$) and point 4 ($x_1 + x_2 = 0$) lie strictly outside it. Removing them would not move the boundary.

(b) Boundary. Each support vector satisfies $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) = 1$:

$$\text{pt } 1\text{: } w_1 + w_2 + b = 1, \qquad \text{pt } 5\text{: } w_1 + b = -1, \qquad \text{pt } 6\text{: } w_2 + b = -1.$$

Points 5 and 6 give $w_1 = w_2 = -1 - b$. Substituting into point 1 gives $-2 - b = 1$, so

$$b = -3, \quad w_1 = w_2 = 2, \qquad 2x_1 + 2x_2 - 3 = 0 \quad (x_1 + x_2 = 1.5)$$

Check: $y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) = (1, 3, 3, 3, 1, 1)$ for points 1–6, so every constraint holds, with equality exactly at 1, 5 and 6. The multipliers $\alpha_1 = 4$ and $\alpha_5 = \alpha_6 = 2$ (all others 0) satisfy $\sum_i \alpha_i y_i = 4 - 2 - 2 = 0$ and $\sum_i \alpha_i y_i \boldsymbol{x}_i = \boldsymbol{w}$ with $\alpha \geq 0$, so the KKT conditions hold and this is the maximum-margin line.

Geometric margin (distance from the boundary to the closest training points):

$$\gamma = \frac{1}{\|\boldsymbol{w}\|} = \frac{\sqrt{2}}{4} \approx$$

0.3536

For example, point 1 is at $|2 + 2 - 3| \,/\sqrt{8} = 0.3536$. Separately, the **full width** between the supporting lines $x_1 + x_2 = 1$ and $x_1 + x_2 = 2$ is $2/\|\boldsymbol{w}\| = 1/\sqrt{2} \approx 0.7071$.

```
$ python assignment3_template.py --task svm
Support vector point IDs: [1, 5, 6]
Weights: (2.0, 2.0); bias: -3.0; geometric margin: 0.35355339059327373
```

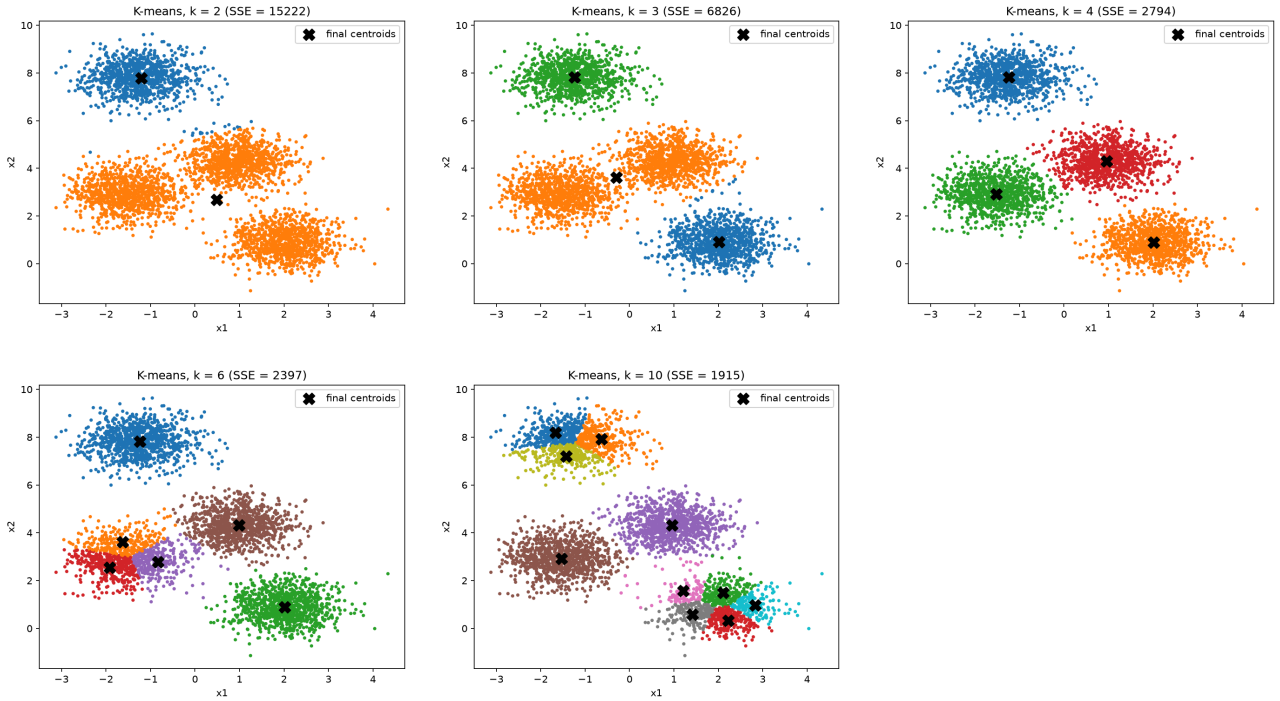
Question 2: K-means

(a) Update steps. `update_assignments` computes the (n, k) matrix of squared Euclidean distances by broadcasting and takes the row-wise `argmin`. **Ties:** `np.argmin` returns the first minimum, so a point that is equally far from several centroids goes to the one with the lowest index. `update_centroids` sets each nonempty cluster's centroid to the mean of its points. **Empty clusters** keep their previous centroid. The function writes into a float copy, so

the input array is not modified. Together, the two rules also handle duplicate initial centroids: only the lowest-index copy receives points, and the other copies stay empty and keep their positions. (The seed-0 initializations used here drew no duplicates.)

(b) Results from `python kmeans_template.py --seed 0 --max-iterations 100`. One seeded generator draws the initial centroids for $k = 2, 3, 4, 6, 10$ in that order. The true blob labels are not used.

k	SSE	cluster sizes
2	15222.2	1017, 2983
3	6826.1	1008, 1992, 1000
4	2794.2	1000, 998, 1002, 1000
6	2396.8	1000, 299, 997, 407, 319, 978
10	1914.8	365, 350, 253, 261, 988, 997, 123, 213, 285, 165



Effect of k. With $k < 4$, K-means has to merge generating groups. At $k = 2$ the upper-left blob gets its own cluster, and the other three share one centroid that sits in the empty space between them. At $k = 3$ the lower-left and middle blobs are merged. At $k = 4$ the clusters match the four generating groups almost exactly (sizes 1000/998/1002/1000 for blobs of 1000 points each). For $k > 4$ the extra centroids split real blobs into arbitrary pieces instead of finding new structure. At $k = 6$ the lower-left blob is cut into three. At $k = 10$ the upper-left blob is cut into three and the lower-right blob into five, while the other two blobs stay whole, so which blobs get split depends on the initialization. SSE always decreases with k, but it falls sharply up to $k = 4$ (15222 $\rightarrow$ 6826 $\rightarrow$ 2794) and only slowly after that (2397, 1915). This elbow sits at the true number of groups.

Question 3: Noisy-label SVM and K-NN repair

Setup (`python noisy_classification_template.py --repeats 20 --seed 0`). One generator seeded with 0 draws the training set, then the independent test set, then the label noise for each trial. In each trial the baseline and the repair use the same features and the same corrupted labels. The repair sees only the training features and the noisy labels. Test labels stay clean and are used only for scoring. Both methods train an RBF SVM with $C = 1$ and $\gamma = 0.01$.

(a) Label noise. Each example gets one uniform draw u_i. It is flipped if $u_i < 0.35$ when its original label is $+1$, or if $u_i < 0.20$ when its original label is -1. The probability is chosen from the original label and there is a single draw per example, so no label can flip twice. The function returns a new array.

(b) K-NN repair. The method builds the full pairwise distance matrix of the training features and sets the diagonal to ∞, so that a point is not its own neighbor. A stable `argsort` of each row then gives the five nearest other points, with equal-distance ties going to the lower row index. The repaired label is the sign of the sum of those five noisy labels (five ± 1 votes cannot tie). Every vote reads the original noisy array, so row order does not change the result.

trial	baseline	K-NN	gain	trial	baseline	K-NN	gain
1	0.8345	0.8615	+0.0270	11	0.8220	0.8600	+0.0380
2	0.8330	0.8720	+0.0390	12	0.8190	0.8645	+0.0455
3	0.8440	0.8675	+0.0235	13	0.8470	0.8645	+0.0175
4	0.8445	0.8700	+0.0255	14	0.8415	0.8715	+0.0300
5	0.8395	0.8725	+0.0330	15	0.8160	0.8715	+0.0555
6	0.8410	0.8715	+0.0305	16	0.8285	0.8485	+0.0200
7	0.8405	0.8690	+0.0285	17	0.8235	0.8695	+0.0460
8	0.7930	0.8480	+0.0550	18	0.8430	0.8700	+0.0270
9	0.8240	0.8615	+0.0375	19	0.7840	0.8335	+0.0495
10	0.8425	0.8715	+0.0290	20	0.7980	0.8690	+0.0710

mean baseline accuracy	mean K-NN repair accuracy	difference
0.8279	0.8644	**+0.0364**

Did it help? Yes. K-NN repair improved clean-test accuracy in all 20 trials, by 0.0175 to 0.0710. It did not reduce the number of wrong labels, but it moved them. Deep inside a class, if all five neighbors truly share the point's class, the vote is wrong only with probability $P(\mathrm{Bin}(5, 0.35) \geq 3) = 0.235$ for positives and $P(\mathrm{Bin}(5, 0.20) \geq 3) = 0.058$ for negatives, instead of 0.35 and 0.20. In the overlap region the neighbors come from both classes, so voting makes labels locally uniform and adds errors there. For this nearly linear SVM ($\gamma = 0.01$), the costly errors are the mislabeled points far from the boundary. Each one has a large hinge loss and pulls the boundary into the wrong region, and because positives are flipped more often, the baseline is pushed toward predicting -1. Moving the errors into the overlap, where the classes are already mixed, gives a cleaner boundary. Repair can hurt when the noise is a local majority. Voting then reinforces the wrong label and erases real fine structure near the boundary.

Acknowledgment of outside help

An AI assistant was used to help convert the original manuscript into this Typst report.